

GraphMind: Efficiently Exploring Multi-Agent LLM Topologies with Graph Neural Networks

Vito Levstik and Gal Zmazek

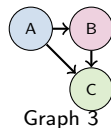
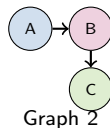
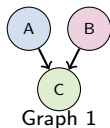
University of Ljubljana

January 11, 2026

The Challenge: Topology Search is Expensive

The Problem:

- Multi-agent LLM systems need configuration
- Which agents? How to connect them?
- **Combinatorial explosion:** millions of possible graphs
- Each evaluation requires many LLM API calls
- **Cost: \$\$\$ + Time**



... millions more ...

Which one is best?

Our Solution:

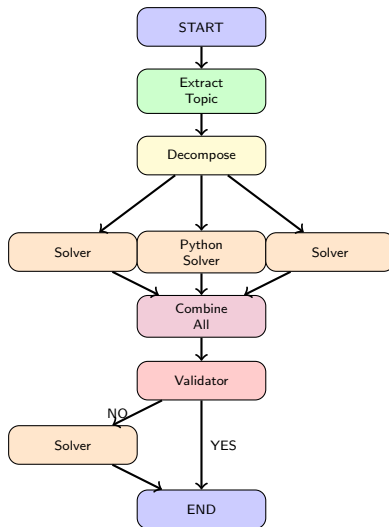
Use GNNs as surrogate models to predict performance

Multi-Agent System for Mathematical Problem Solving

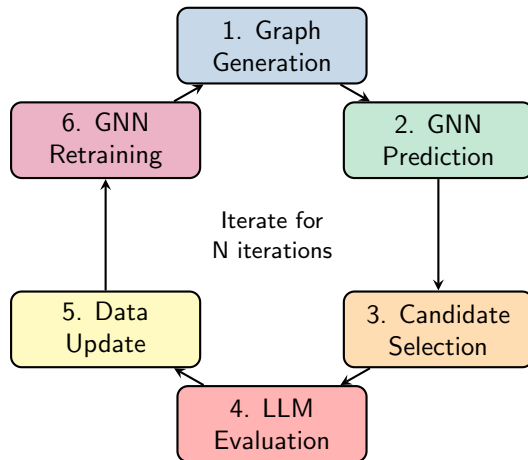
8 Specialized Agent Types:

- **Solver**: Generate solutions
- **Python_solver**: Execute code
- **Validator**: Check correctness
- **Extract_topic**: Identify concepts
- **Decompose**: Break into subproblems
- **Split**: Parallel exploration
- **Combine_all**: Merge results
- **Explain**: Provide reasoning

Task: Solve math problems from NVIDIA OpenMathInstruct-1 dataset



GraphMind Pipeline: 6-Step Iterative Process

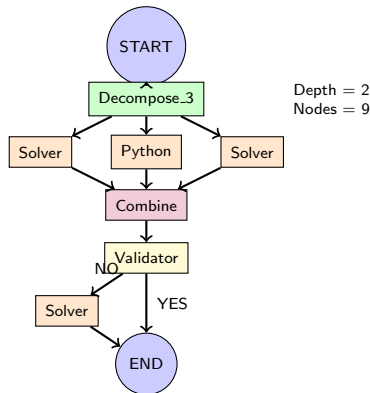


Key Idea: Generate many candidates GNN filters Evaluate only the best Improve GNN

Step 1: Graph Generation

Semi-Random Generation:

- Generate 200,000 graphs per iteration
- Constraints: max 8 nodes, max depth 3
- Rules enforce structure:
 - Decompose_3 3 branches
 - Combine_all merges branches
 - START and END nodes
- Filter duplicates
- Result: 1,000-2,000 unique graphs



Example generated graph

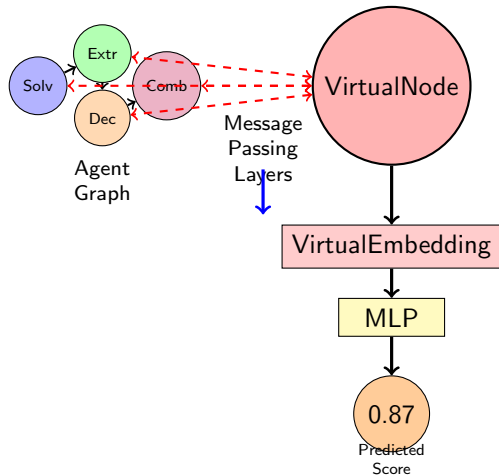
Step 2: GNN Prediction with Virtual Node

GNN Architecture:

- **Input:** One-hot node features (15 types)
- **Models:** GCN, GraphSAGE, GAT
- **Virtual Node:** Connected to all nodes
- **Message Passing:** 2-4 layers
- **Output:** Score $[0,1]$ via sigmoid

Virtual Node Technique:

- Aggregates global graph information
- Final embedding MLP prediction
- Enables graph-level predictions

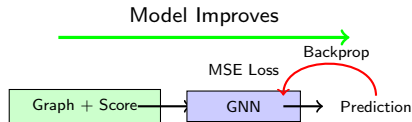


GNN with virtual node architecture

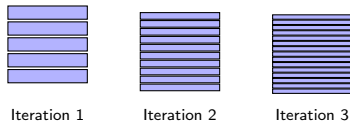
Steps 5-6: Data Update and GNN Retraining

Step 5: Data Update

- Add evaluated graphs to training set
- Each graph: structure + true score
- Dataset grows by 5 graphs/iteration
- Accumulates over iterations



Training Data Growth



Step 6: GNN Retraining

- Retrain every iteration
- Supervised learning: MSE loss
- Adam optimizer
- 300 epochs, batch size 32
- Model learns structure-performance mapping

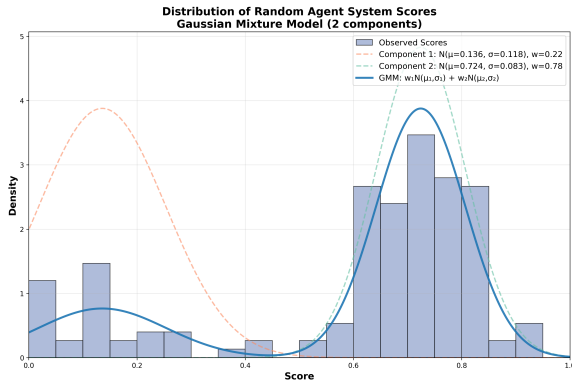
Random Baseline: Understanding the Distribution

Establishing a Baseline:

- Generated 5M random graphs
- Sampled 150 for evaluation
- Measured performance on math problems
- Analyzed score distribution

Observed: Bimodal Distribution

- Two distinct clusters
- Good graphs (> 0.5 accuracy)
- Poor graphs (< 0.3 accuracy)
- Simple distributions insufficient



Random graph distribution fitted with GMM
Two clusters: good and poor performing graphs

Solution: Gaussian Mixture Model

GNN Models: Architecture Details

Three GNN Architectures:

① GCN (Graph Convolutional Network)

- Simple spectral convolution
- Fast and efficient

② GraphSAGE

- Neighborhood sampling
- Inductive learning

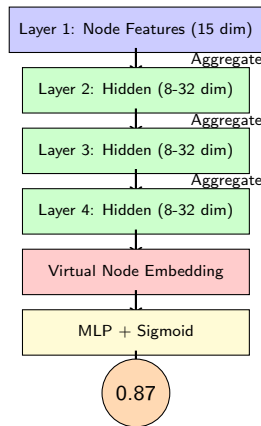
③ GAT (Graph Attention Network)

- Attention mechanisms
- Weighted aggregation

Hyperparameter Search:

- Layers: [2, 3, 4]
- Hidden dim: [4, 8, 16, 32]
- Dropout: [0.0, 0.1, 0.2, 0.5]

Message Passing Layers



Results: GNN Model Performance

Hyperparameter Search Results:

Model	Layers	Hidden	MSE
GCN	4	8	0.0123
GraphSAGE	4	16	0.0089
GAT	4	8	0.0156

Test Performance (10 runs):

Model	Test MSE	
GCN	0.0145	0.0021
GraphSAGE	0.0102	0.0018
GAT	0.0178	0.0025

GraphSAGE performs best!

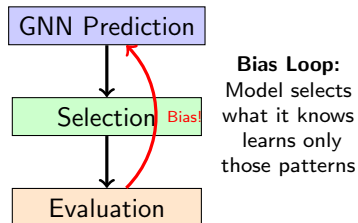
Key Insights:

- 4 layers optimal
- Hidden dim 8-16 works well
- GraphSAGE generalizes best
- Low variance across runs

Pipeline Dynamics: The Prediction Paradox (1/2)

An Interesting Phenomenon:

- GNN predictions become very accurate
- But... this creates a bias!
- Model overfits to its own selections
- Training data becomes skewed



The Feedback Loop:

- 1 GNN predicts good graphs
- 2 Only predicted "good" graphs evaluated
- 3 Training data biased toward these
- 4 GNN learns to predict "more of same"
- 5 Diversity decreases over iterations

Result: Training data becomes increasingly biased toward model's preferred patterns

Pipeline Dynamics: The Prediction Paradox (2/2)

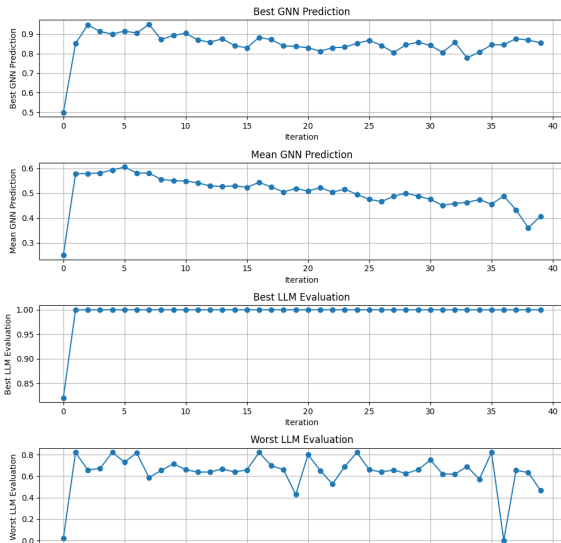
Observable in Metrics:

- RMSE appears to improve
- But on biased sample!
- GNN score variance increases
- Actual diversity decreases

Key Insight:

- Model becomes "too good" at its own strategy
- Reduces exploration of search space
- Selection bias accumulates

Despite this limitation:



Results: GNN-Guided vs Random Search

Experimental Setup:

- **Baseline:** 150 random graphs
- **GraphMind:** 30 iterations
- Each graph evaluated on 10 problems
- Modeled random baseline as GMM

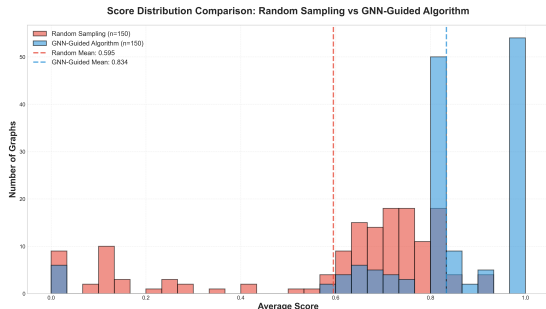
Key Results:

- **GraphSAGE:** Best average (0.72 vs 0.43)
- **GAT:** 10% of graphs scored 1.0
- **GCN:** Strong performance (0.69 avg)
- All models significantly beat random

Cost Reduction:

Evaluate 150 instead of 6M graphs

Score Distributions



Random baseline (gray) vs GNN-guided search (colored)

Challenges and Future Work

Current Limitations:

- Selection bias feedback loop
- Limited to math problems
- High variance in LLM evaluations
- Fixed evaluation budget

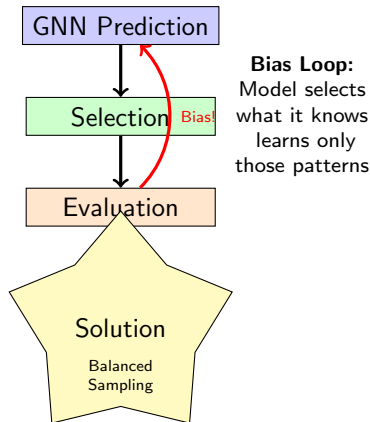
Future Directions:

1 Mitigate Bias:

- Stratified sampling
- Diversity-aware selection
- Exploration-exploitation balance

2 Smart Generation:

- Learn from failures
- Exclude similar bad graphs



Key Takeaways:

- GNNs can effectively predict multi-agent system performance
- Iterative refinement improves predictions over time
- Massive cost savings enable practical topology search
- GraphSAGE architecture works best for this task

GitHub: github.com/vitolev/GraphMind

Thank you! Questions?